

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Algoritmi I

Appunti a cura di
Lorenzo Ferraro

12 settembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Algoritmi I*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Introduzione	1
1.1	Gli Algoritmi	1
1.2	Efficienza	1
1.2.1	Tempo	1
1.2.2	Spazio	2
2	Costo computazionale	3
2.1	Notazione asintotica	3
2.1.1	Notazione asintotica O	3
2.1.2	Notazione asintotica Ω	4
2.1.3	Notazione asintotica Θ	4

Capitolo 1

Introduzione

1.1 Gli Algoritmi

La definizione di informatica proposta dall'ACM (Association for Computing Machinery) è:
"L'informatica è la scienza degli algoritmi che descrivono e trasformano l'informazione: la loro teoria, analisi, progetto, efficienza, realizzazione e applicazione."

Ma cos'è un algoritmo?

Definizione 1.1 – Algoritmo

Un **algoritmo** è una sequenza di comandi *elementari* e *univoci* che terminano in un tempo finito e che operano su *strutture dati*.

Un comando è *elementare* quando non può essere scomposto in comandi più semplici, mentre un comando è *univoco* quando può essere interpretato in un solo modo. Le *strutture dati* invece sono degli strumenti necessari al fine di gestire i dati veri e propri, consentendoci di leggerli e modificarli. Ci sono diverse strutture dati e non ne esiste una universale per tutti i problemi: per questo, in base al proprio problema bisogna scegliere la struttura dati più coerente.

1.2 Efficienza

1.2.1 Tempo

Un algoritmo viene sempre eseguito in un tempo finito. Ciò però non significa che tutti gli algoritmi che rilasciano un output corretto siano dei buoni algoritmi. Infatti, uno degli aspetti più importanti quando si parla di un algoritmo è la sua **efficienza**, ovvero il tempo di esecuzione dell'algoritmo e la sua memoria richiesta (il tempo di esecuzione ha però spesso la priorità). Si potrebbe, però, pensare che un supercomputer molto veloce possa bilanciare la lentezza di un algoritmo, ma ciò è errato. Vediamo perché con un esempio.

Prendiamo due computer V e L, dove V esegue 10^9 operazioni al secondo mentre L ne esegue 10^7 , e abbiamo due algoritmi IS e MS, che rispettivamente richiedono di eseguire n^2 e $50n \log n$ operazioni, con n che indica la dimensione dell'input. Facciamo ora eseguire al computer veloce

V l'algoritmo più lento IS e al computer lento IS l'algoritmo più veloce MS, ponendo come input $n = 10^6$. Vediamo come si comportano:

- $V(IS) = \frac{2(10^6)^2}{10^9} = 2000s$
- $L(MS) = \frac{50 \cdot 10^6 \log 10^6}{10^7} = 100s$

È facile notare come il computer più lento che esegue l'algoritmo più veloce sia una soluzione ben migliore rispetto al computer più veloce che esegue l'algoritmo più lento. Inoltre, più il valore di input n aumenta, più questa differenza si fa evidente.

1.2.2 Spazio

Nell'esecuzione di un algoritmo, questo crea anche diverse risorse in memoria che occupano spazio. Questo spazio viene principalmente istanziato nella **memoria principale**, ossia la RAM. Ma, se non si ha abbastanza spazio nella RAM, si passa ad allocare queste risorse nella **memoria secondaria**, ossia nell'SSD. Il dover accedere a questa memoria rende però molto più lento l'algoritmo, dato che ci sono sia dati sulla RAM che sull'SSD, e se c'è bisogno di fare un confronto tra loro, questo sarà lentissimo. Per questo, è necessario cercare di evitare algoritmi che consumino troppo spazio, altrimenti il rischio è quello di finire la RAM a disposizione.

Capitolo 2

Costo computazionale

2.1 Notazione asintotica

Cerchiamo ora un metodo efficace per poter confrontare due algoritmi che svolgono lo stesso lavoro, per poter capire quale dei due sia il più efficiente. Questa valutazione che faremo è utile, però, solo quando si parla di input molto grandi.

Prima di iniziare a confrontare algoritmi, bisogna conoscere la notazione asintotica.

Definizione 2.1 – Notazione asintotica

A livello matematico, la **notazione asintotica** è uno strumento che permette di confrontare il tasso di crescita di una funzione rispetto a un'altra. Esistono 3 tipi di notazione asintotica:

- 1) Notazione asintotica $\mathbf{O}$, che ci permette di calcolare il limite asintotico *superiore*
- 2) Notazione asintotica $\mathbf{\Omega}$, che ci permette di calcolare il limite asintotico *inferiore*
- 3) Notazione asintotica $\mathbf{\Theta}$, che ci permette di calcolare il limite asintotico *stretto*

N.B. Prima di passare allo studio di queste tre diverse tipologie di notazione asintotica chiariamo bene una cosa: scrivere $f(n) = O(g(n))$ e scrivere $f(n) \in O(g(n))$ è la stessa identica cosa. Discorso analogo anche per le notazioni Ω e Θ .

2.1.1 Notazione asintotica $\mathbf{O}$

Definizione 2.2 – Notazione asintotica $\mathbf{O}$

Date due funzioni $f(n), g(n) \geq 0$, si dice che $f(n) = \mathbf{O}(n)$ (o $f(n) \in O(n)$) se $\exists c > 0$ e $\exists n_0 \in \mathbb{N} : 0 \leq f(n) \leq c g(n) \ \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $f(n)$ si trova sempre **sotto** la funzione $g(n)$.

Esempi

$$f(n) = 3n + 3$$

- $f(n) = \mathbf{O}(n^2)$ dato che, se prendiamo $c = 1$, $3n + 3 \leq n^2 \ \forall n \geq 4$

- $f(n) = O(n^2)$ dato che, se prendiamo $c = 4$, $3n + 3 \leq 4n \quad \forall n \geq 3$

$f(n) = \sum_{i=0}^m a_i n^i$ con $a_m \geq 0$ (polinomio di grado m)

$f(n) = O(n^m)$ dato che $f(n) = \sum_{i=0}^m a_i n^i \leq \sum_{i: c \cdot a_i \geq 0} a_i n^i \leq n^m \sum_{i: c \cdot a_i \geq 0} a_i$, e se poniamo $c = \sum_{i: c \cdot a_i \geq 0} a_i$, troviamo proprio che $f(n) \leq cn^m \quad \forall n \geq 1$.

Proprietà 2.1 – Regole generali della notazione asintotica O

- $\log_a n = O(n^{\frac{1}{b}})$, ossia un qualsiasi logaritmo è dominato da una qualunque radice
- $n^{\frac{1}{a}} = O(n^b)$ (con $a, b \geq 1$), ossia una qualsiasi radice è dominata da un qualunque polinomio
- $n^a = O(b^n)$ (con $b > 1$), ossia un polinomio è dominato da un qualunque esponenziale

2.1.2 Notazione asintotica Ω

Definizione 2.3 – Notazione asintotica Ω

Date due funzioni $f(n), g(n) \geq 0$, si dice che $f(n) = \Omega(n)$ (o $f(n) \in \Omega(n)$) se $\exists c > 0$ e $\exists n_0 \in \mathbb{N} : 0 \leq cg(n) \leq f(n) \quad \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $f(n)$ si trova sempre **sopra** la funzione $g(n)$.

2.1.3 Notazione asintotica Θ

Definizione 2.4 – Notazione asintotica Θ

Date due funzioni $f(n), g(n) \geq 0$, si dice che $f(n) = \Theta(n)$ (o $f(n) \in \Theta(n)$) se $\exists c_1, c_2 > 0$ e $\exists n_0 \in \mathbb{N} : c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$, ossia se da una certa n_0 in poi la funzione $f(n)$ si trova sempre **tra** le funzioni $c_1 g(n)$ e $c_2 g(n)$.

N.B. Per dimostrare che $f(n) = \Theta(g(n))$ basta dimostrare che $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.